
APLICACIONES WEB

UNIDAD II

Herramientas de Programación Web del Servidor

Desarrollo completo con **Java**: Servlets, JSP y Spring Boot

Conceptos · características · ejemplos cortos y completos

Código *frontend* y *backend* · pantallas de ejecución

Universidad Técnica Estatal de Quevedo

Facultad de Ciencias de la Computación

Carrera de Ingeniería de Software (Rediseño)

Quinto nivel — Período Académico 2026–2027

Resultado de aprendizaje (RAU 2): desarrollar aplicaciones web dinámicas del lado del servidor con Java, implementando lógica de negocio, procesamiento de formularios y mecanismos de seguridad.

Sobre esta unidad. La Unidad II del curso de Aplicaciones Web aborda la *programación del lado del servidor*: el conjunto de tecnologías que reciben las peticiones del navegador, ejecutan lógica de negocio, consultan bases de datos y devuelven respuestas dinámicas. Aunque el sílabo presenta varias plataformas (PERL, PHP, ASP.NET, Java), este documento desarrolla la unidad **íntegramente en Java**, por ser el lenguaje dominante del software empresarial y el que mejor articula los prerrequisitos de Estructura de Datos y Base de Datos. El recorrido va desde el modelo histórico (Servlets y JSP) hasta el estándar moderno (Spring Boot 3), construyendo una aplicación completa con interfaz de usuario (*frontend*) y lógica de servidor (*backend*).

Las “capturas de pantalla” que ilustran la ejecución se han reconstruido de forma fiel y reproducible para que el estudiante reconozca con exactitud lo que verá en su propia máquina al ejecutar cada ejemplo en *IntelliJ IDEA Ultimate* con *Java 21* y *Spring Boot 3.4*.

Índice general

1. Fundamentos de la programación del lado del servidor con Java	1
1.1. ¿Qué significa «lado del servidor»?	1
1.1.1. El modelo petición–respuesta de HTTP	2
1.1.2. Arquitectura de una aplicación web Java	2
1.2. Por qué Java en el servidor	3
1.2.1. Ventajas técnicas de la plataforma	3
1.2.2. Treinta años de Java en la web	3
1.3. Los tres modelos que veremos	4
2. Servlets: el modelo base de Java web	5
2.1. ¿Qué es un Servlet?	5
2.1.1. Definición y paquete <code>jakarta.servlet</code>	5
2.1.2. Ciclo de vida de un Servlet	5
2.2. Ejemplo corto: «Hola» con <code>doGet</code>	6
2.3. Ejemplo completo: formulario con <code>doGet</code> y <code>doPost</code>	7
2.4. Características y limitaciones del modelo Servlet	10
3. JSP, EL y JSTL: vistas dinámicas	12
3.1. JSP: la página que se convierte en Servlet	14
3.1.1. Del JSP al Servlet: el ciclo de traducción	14
3.1.2. Directivas: configurar la página	14
3.2. EL: el <i>Expression Language</i>	14
3.2.1. Sintaxis y acceso a los datos	15
3.2.2. Ejemplo corto: leer parámetros con EL	15
3.3. JSTL: la biblioteca estándar de etiquetas	16
3.3.1. Etiquetas de iteración y condición	16
3.3.2. Ejemplo corto: recorrer una lista con <code>c:forEach</code>	16
3.4. Ejemplo completo: listado de cursos con MVC	17
3.4.1. Diseño: el patrón MVC en tres roles	17
3.4.2. Implementación: modelo, controlador y vista	17
3.4.3. Ejecución y resultado	19
4. Spring Boot: el estándar moderno	20

4.1. ¿Qué resuelve Spring Boot?	20
4.1.1. Las tres ideas centrales	20
4.1.2. Inyección de dependencias en una frase	21
4.2. Estructura de un proyecto Spring Boot	21
4.3. Ejemplo corto: primer endpoint REST	22
4.4. Configuración con <code>application.properties</code>	24
5. Aplicación completa: CRUD con frontend y backend	25
5.1. Qué vamos a construir	25
5.2. Backend	26
5.2.1. Dependencias y configuración	26
5.2.2. Modelo: la entidad <code>Producto</code>	27
5.2.3. Repositorio	28
5.2.4. Servicio	28
5.2.5. Controlador web	29
5.2.6. Datos de prueba	30
5.3. Frontend	31
5.3.1. Vista de listado	31
5.3.2. Vista de formulario	32
5.4. Ejecución	33
5.5. Inspección de la base de datos	35
6. API REST y consumo desde el cliente	36
6.1. De vistas a datos: ¿qué cambia?	36
6.1.1. HTML armado frente a datos JSON	36
6.1.2. Verbos HTTP y códigos de estado	37
6.2. Backend: el controlador REST	37
6.3. Frontend: consumir la API con <code>fetch</code>	39
7. Síntesis, buenas prácticas y ejercicios	41
7.1. Comparativa de los tres modelos	41
7.2. Buenas prácticas profesionales	42
7.3. Ejercicios propuestos en IntelliJ IDEA	42
7.4. Cierre de la unidad	43
Bibliografía	45

Capítulo 1

Fundamentos de la programación del lado del servidor con Java

«*Write once, run anywhere*. Java cumplió la promesa en el servidor: tres décadas después sigue siendo la columna vertebral de la banca, la salud y el gobierno digital del planeta.»

— Adaptado de la promesa original de Sun Microsystems

Resultado de aprendizaje del capítulo

Al concluir este capítulo el estudiante explica el modelo petición–respuesta de HTTP, describe la arquitectura de una aplicación web del lado del servidor en Java y justifica por qué conviven hoy varios modelos de programación (Servlets, JSP y Spring Boot) dentro del mismo ecosistema [1].

1.1 ¿Qué significa «lado del servidor»?

En la Unidad I el estudiante construyó interfaces que se ejecutan en el **navegador** del usuario (HTML5, CSS3 y JavaScript): el llamado *lado del cliente*. La Unidad II cruza al otro extremo de la conversación, el **lado del servidor** (*server-side*): el programa que se ejecuta en una máquina remota, recibe la petición del navegador, decide qué hacer con ella —validar datos, consultar una base, aplicar reglas de negocio— y devuelve una respuesta. El cliente nunca ve ese código; solo ve su resultado.

Cliente vs. servidor en una frase

El **cliente** pregunta y muestra; el **servidor** decide y responde. El navegador no tiene acceso a la base de datos ni a la lógica sensible: esa es, precisamente, la razón de existir del lado del servidor.

1.1.1 El modelo petición–respuesta de HTTP

Toda la web funciona sobre un ciclo simple y repetido: el cliente envía una **petición** (*request*) y el servidor devuelve una **respuesta** (*response*). Este modelo es *sin estado* (*stateless*): cada petición es independiente y el servidor no recuerda, por sí mismo, lo ocurrido en peticiones anteriores. La Figura 1.1 sintetiza el ciclo.

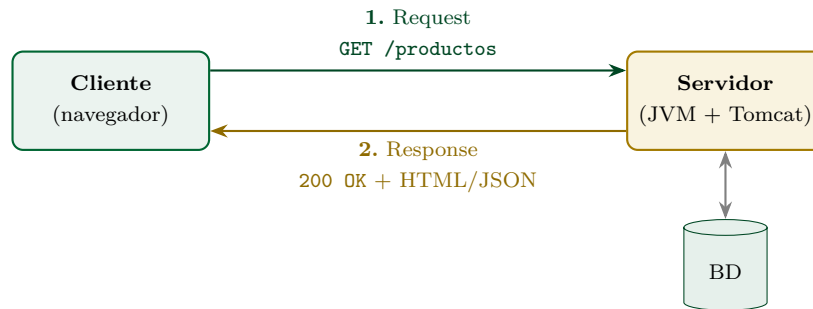


Figura 1.1: Modelo petición–respuesta: el cliente solicita un recurso y el servidor lo produce, normalmente consultando una base de datos.

Como se observa en la Figura 1.1, el servidor puede apoyarse en una base de datos para construir la respuesta. El navegador no participa de ese diálogo interno: recibe ya el HTML o el JSON terminado.

1.1.2 Arquitectura de una aplicación web Java

Una petición que llega a una aplicación Java atraviesa varias capas antes de producir una respuesta. La Figura 1.2 muestra ese recorrido: el navegador habla con un servidor HTTP (Apache o Nginx, opcional como *proxy*), que reenvía al **contenedor de servlets** (Tomcat, Jetty), donde se ejecuta el *bytecode* sobre la **Máquina Virtual de Java** (JVM).

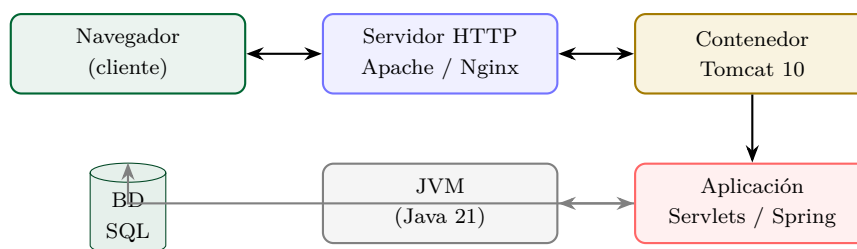


Figura 1.2: Arquitectura por capas de una aplicación web Java: el contenedor de servlets ejecuta la aplicación sobre la JVM y media entre la red y la lógica.

El **contenedor de servlets** (Figura 1.2) es la pieza clave: traduce los *bytes* de la red en objetos Java (`HttpServletRequest`) y convierte los objetos de respuesta en *bytes* HTTP. Frameworks como Spring Boot incluyen un Tomcat *embebido*, de modo que la aplicación se ejecuta con un solo comando sin instalar un servidor aparte.

1.2 Por qué Java en el servidor

Java domina el desarrollo web empresarial por razones concretas, no por inercia. Esta sección lo justifica desde dos ángulos complementarios: las ventajas técnicas de la plataforma y la trayectoria histórica que la consolidó.

1.2.1 Ventajas técnicas de la plataforma

La caja siguiente resume sus características como plataforma de servidor.

Características de Java como plataforma de servidor

- **Portabilidad:** el mismo *bytecode* corre en Windows, Linux o macOS sobre cualquier JVM.
- **Tipado estático fuerte:** muchos errores se detectan al compilar, no en producción.
- **Concurrencia madura:** la JVM gestiona miles de peticiones simultáneas con *threads* (y *virtual threads* desde Java 21).
- **Ecosistema gigantesco:** Spring, Hibernate, Jakarta EE, millones de librerías en Maven Central.
- **Rendimiento:** el compilador JIT optimiza el código en caliente hasta velocidades cercanas a C.
- **Soporte de largo plazo (LTS):** versiones estables mantenidas por años (Java 17, 21, 25).

1.2.2 Treinta años de Java en la web

De los Applets a Spring Boot

James Gosling y su equipo en Sun Microsystems crearon Java en 1995. Sun introdujo los **Applets** (Java en el navegador, 1996) y los **Servlets** (Java en el servidor, 1997). En 1999 llegó **J2EE**, potente pero pesado: un proyecto exigía decenas de archivos XML. La frustración generó dos respuestas: **Spring Framework** (Rod Johnson, 2003), que simplificó la inyección de dependencias, y **Spring Boot** (2014), que eliminó la configuración manual con el principio *convención sobre configuración*. Hoy Spring Boot es el framework Java dominante en aplicaciones web empresariales [6].

La Tabla 1.1 sintetiza las versiones de Java y Spring relevantes en 2026 y el aporte de cada una.

Tabla 1.1: Versiones de Java y Spring relevantes en 2026.

Año	Versión	Aporte principal
2014	Java 8	Lambdas, <i>streams</i> , <code>Optional</code> .
2014	Spring Boot 1.0	Convención sobre configuración.
2018	Java 11 LTS	Primera LTS del nuevo ciclo de seis meses.
2021	Java 17 LTS	<i>Records</i> , <i>sealed classes</i> , <i>pattern matching</i> .
2022	Spring Boot 3.0	Migración a Jakarta EE 9 (paquetes <code>jakarta.*</code>).
2023	Java 21 LTS	<i>Virtual threads</i> , colecciones secuenciadas.
2024	Spring Boot 3.4	Compatible con Java 21+, observabilidad mejorada.
2025	Java 25 LTS	Patrones de cadena en <code>switch</code> .

La Tabla 1.1 explica un hecho que desconcierta a los principiantes: en Java conviven varios modelos de programación web simultáneamente. Los tres que desarrollaremos —Servlets, JSP y Spring Boot— no son rivales sino capas históricas: Spring Boot se apoya internamente en Servlets, y comprender el modelo base ilumina el moderno.

1.3 Los tres modelos que veremos

La Tabla 1.2 anticipa el itinerario del documento y sitúa cada modelo en su contexto de uso.

Tabla 1.2: Los tres modelos de Java web desarrollados en esta unidad.

Modelo	Idea central	Capítulo
Servlets	Clase Java que responde a HTTP; el “motor” de todo.	Cap. 2
JSP + EL/JSTL	Plantillas HTML con datos dinámicos del servidor.	Cap. 3
Spring Boot	Framework opinado que automatiza la configuración.	Cap. 4

Con los fundamentos de la Tabla 1.2 claros, el Capítulo 2 abre el motor por dentro: el Servlet.

Capítulo 2

Servlets: el modelo base de Java web

«Antes de aprender el atajo conviene conocer el camino: el Servlet es el camino que todo framework Java recorre por debajo.»

— Nota docente

Resultado de aprendizaje del capítulo

Al concluir este capítulo el estudiante implementa Servlets que atienden peticiones **GET** y **POST**, explica el ciclo de vida gestionado por el contenedor y procesa formularios con validación del lado del servidor.

2.1 ¿Qué es un Servlet?

Antes de escribir código conviene fijar dos cosas: qué es exactamente un Servlet y bajo qué paquete vive hoy, y cómo el contenedor lo gestiona durante su existencia.

2.1.1 Definición y paquete `jakarta.servlet`

Servlet

Un **Servlet** es una clase Java que extiende `HttpServlet` y que el contenedor (Tomcat) instancia para atender peticiones HTTP. Cada método (`doGet`, `doPost`, ...) responde a un verbo HTTP y recibe dos objetos: la petición (`HttpServletRequest`) y la respuesta (`HttpServletResponse`).

Desde Spring Boot 3 y Jakarta EE 9 las clases viven en el paquete `jakarta.servlet.*` (antes `javax.servlet.*`). Es el cambio más visible al migrar proyectos antiguos.

2.1.2 Ciclo de vida de un Servlet

El contenedor —no el programador— controla cuándo se crea, se usa y se destruye un Servlet. La Figura 2.1 muestra las tres fases.

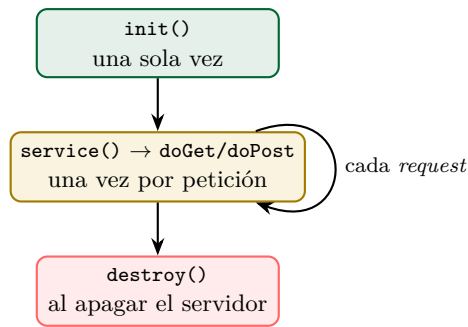


Figura 2.1: Ciclo de vida de un Servlet: `init` se ejecuta una vez, `service` (que delega en `doGet/doPost`) una vez por petición, y `destroy` al detener el contenedor.

La consecuencia práctica de la Figura 2.1 es importante: como una sola instancia atiende muchas peticiones concurrentes, **los campos de instancia no deben guardar datos de un usuario**, porque se compartirían entre todos.

2.2 Ejemplo corto: «Hola» con `doGet`

Ejemplo 2.1 (RESUELTO) — Primer Servlet

Enunciado. Crear un Servlet que responda con un saludo personalizado al parámetro nombre de la *query string*.

```

1 package ec.edu.uteq.unidad2;                // paquete del
   ↪ archivo
2
3 import jakarta.servlet.annotation.WebServlet; // mapeo declarativo
   ↪ de la URL
4 import jakarta.servlet.http.*;              // request,
   ↪ response, servlet
5 import java.io.IOException;
6
7 @WebServlet("/hola")                         // este servlet
   ↪ responde a /hola
8 public class HolaServlet extends HttpServlet {
9     @Override
10    protected void doGet(HttpServletRequest req, HttpServletResponse
   ↪ resp)
11        throws IOException {
12        String nombre = req.getParameter("nombre"); // lee ?nombre=...
13        if (nombre == null || nombre.isBlank()) {   // valor por
   ↪ defecto
14            nombre = "Mundo";
15        }
16        resp.setContentType("text/html; charset=UTF-8"); // tipo de
   ↪ respuesta
17        resp.getWriter().write("""
18            <!DOCTYPE html>
  
```

```
19         <html lang="es-EC">
20         <head><meta charset="UTF-8"><title>Saludo</title></head>
21         <body><h1>¡Hola, %s, desde un Servlet!</h1></body>
22         </html>
23         """.formatted(nombre));           // text block +
24         ↪ formatted
25     }
```

Listing 2.1: HolaServlet.java

Al ejecutar el proyecto en Tomcat y visitar la URL con el parámetro, el navegador muestra el saludo. La Figura 2.2 reproduce esa pantalla.



Figura 2.2: Ejecución del Ejemplo 2.1: el Servlet leyó `nombre=Ana` de la URL y devolvió el HTML con el saludo.

2.3 Ejemplo completo: formulario con doGet y doPost

Un Servlet realista combina dos verbos: `doGet` muestra el formulario y `doPost` procesa el envío, valida los datos y responde. El siguiente ejemplo es un registro de estudiantes con validación del lado del servidor.

Ejemplo 2.2 (RESUELTO) — Registro con validación

Enunciado. Construir un Servlet que muestre un formulario de registro (nombre y edad) y, al enviarlo, valide los datos: el nombre no puede estar vacío y la edad debe estar entre 16 y 99. Si hay errores, se muestran; si todo es correcto, se confirma el registro.

Backend — Servlet completo:

```
1 package ec.edu.uteq.unidad2;
2
3 import jakarta.servlet.annotation.WebServlet;
4 import jakarta.servlet.http.*;
5 import java.io.IOException;
6 import java.io.PrintWriter;
7 import java.util.ArrayList;
8 import java.util.List;
9
10 @WebServlet("/registro")
11 public class RegistroServlet extends HttpServlet {
12
```

```

13 // GET: muestra el formulario vacío
14 @Override
15 protected void doGet(HttpServletRequest req, HttpServletResponse
    ↪ resp)
16     throws IOException {
17     responderFormulario(resp, "", "", new ArrayList<>());
18 }
19
20 // POST: valida y responde
21 @Override
22 protected void doPost(HttpServletRequest req, HttpServletResponse
    ↪ resp)
23     throws IOException {
24     String nombre = trim(req.getParameter("nombre"));
25     String edadTxt = trim(req.getParameter("edad"));
26     List<String> errores = new ArrayList<>();
27
28     if (nombre.isBlank()) {
29         errores.add("El nombre es obligatorio.");
30     }
31     int edad = -1;
32     try {
33         edad = Integer.parseInt(edadTxt);
34         if (edad < 16 || edad > 99) {
35             errores.add("La edad debe estar entre 16 y 99.");
36         }
37     } catch (NumberFormatException e) {
38         errores.add("La edad debe ser un número entero.");
39     }
40
41     if (errores.isEmpty()) { // caso de éxito
42         resp.setContentType("text/html; charset=UTF-8");
43         PrintWriter out = resp.getWriter();
44         out.printf("""
45             <!DOCTYPE html><html lang="es-EC"><head>
46             <meta charset="UTF-8"><title>OK</title></head><body>
47             <h1>Registro exitoso</h1>
48             <p>Bienvenido/a <b>%s</b> (%d años).</p>
49             <a href="registro">Registrar otro</a>
50             </body></html>
51             """, escapar(nombre), edad);
52     } else { // re-muestra
53         ↪ con errores
54         responderFormulario(resp, nombre, edadTxt, errores);
55     }
56
57 // Genera el HTML del formulario (reutilizado por GET y POST)
58 private void responderFormulario(HttpServletResponse resp, String
    ↪ nombre,
59     String edad, List<String> errores) throws IOException {

```

```

60     resp.setContentType("text/html; charset=UTF-8");
61     PrintWriter out = resp.getWriter();
62     StringBuilder err = new StringBuilder();
63     for (String e : errores) {
64         err.append("<li>").append(escapar(e)).append("</li>");
65     }
66     out.printf("""
67         <!DOCTYPE html><html lang="es-EC"><head>
68         <meta charset="UTF-8"><title>Registro</title></head><body>
69         <h1>Registro de estudiante</h1>
70         %s
71         <form method="post" action="registro">
72             <p><label>Nombre: <input name="nombre" value="%s"></label>
73             ↪ ></p>
74             <p><label>Edad: <input name="edad" value="%s"></label></p>
75             ↪ >
76             <button type="submit">Registrar</button>
77         </form></body></html>
78         """,
79         errores.isEmpty() ? "" : "<ul style='color:red'>" + err + "
80         ↪ </ul>",
81         escapar(nombre), escapar(edad));
82     }
83
84     private String trim(String s) { return s == null ? "" : s.trim();
85     ↪ }
86
87     private String escapar(String s) {                                     // anti-XSS bá
88     ↪ sico
89         return s.replace("&", "&amp;").replace("<", "&lt;").replace(">", "
90     ↪ &gt;");
91     }
92 }

```

Listing 2.2: RegistroServlet.java

Validar siempre en el servidor

La validación de HTML5 (atributos `required`, `min`, `max`) es cómoda pero **no es seguridad**: el usuario puede saltarla con una petición manual. La validación del servidor del Ejemplo 2.2 es la que realmente protege la aplicación. El método `escapar()` previene además ataques *Cross-Site Scripting* (XSS) al neutralizar etiquetas inyectadas.

La Figura 2.3 muestra el formulario inicial servido por `doGet`; la Figura 2.4 muestra la reacción a datos inválidos, y la Figura 2.5 la confirmación de éxito.



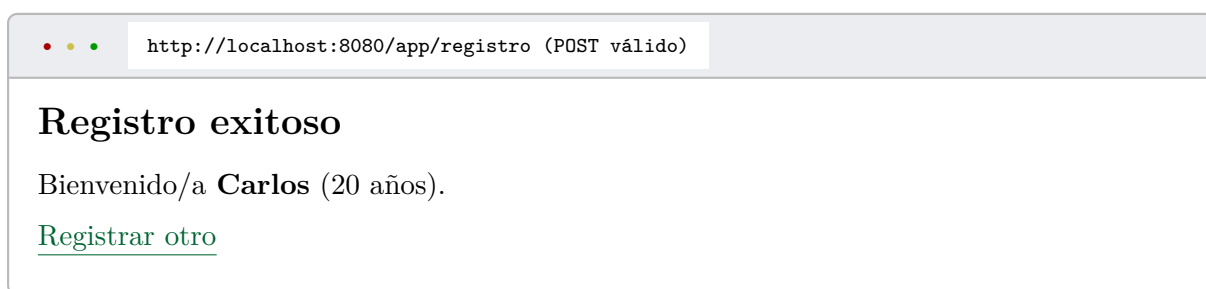
A screenshot of a web browser window. The address bar shows 'http://localhost:8080/app/registro'. The page title is 'Registro de estudiante'. Below the title, there are two input fields: 'Nombre:' followed by an empty text box, and 'Edad:' followed by an empty text box. Below these fields is a button labeled 'Registrar'.

Figura 2.3: Ejecución del Ejemplo 2.2: formulario inicial entregado por doGet.



A screenshot of a web browser window. The address bar shows 'http://localhost:8080/app/registro (POST con edad = 12)'. The page title is 'Registro de estudiante'. Below the title, there is a red error message: '• La edad debe estar entre 16 y 99.' Below the message, there are two input fields: 'Nombre:' followed by a text box containing 'Carlos', and 'Edad:' followed by a text box containing '12'. Below these fields is a button labeled 'Registrar'.

Figura 2.4: Ejecución del Ejemplo 2.2: el servidor rechaza la edad 12 y reimprime el formulario con el mensaje de error, conservando lo escrito.



A screenshot of a web browser window. The address bar shows 'http://localhost:8080/app/registro (POST válido)'. The page title is 'Registro exitoso'. Below the title, there is a message: 'Bienvenido/a Carlos (20 años)'. Below the message is a green link labeled 'Registrar otro'.

Figura 2.5: Ejecución del Ejemplo 2.2: con datos válidos el Servlet confirma el registro.

2.4 Características y limitaciones del modelo Servlet

El recorrido de las Figuras 2.3–2.5 revela tanto la potencia como la incomodidad del modelo: escribir HTML dentro de cadenas Java es tedioso y propenso a errores. La Tabla 2.1 resume sus características.

Tabla 2.1: Características del modelo Servlet puro.

Aspecto	Valoración
Control	Total sobre la petición y la respuesta.
Rendimiento	Muy alto: es la capa más cercana al contenedor.
Productividad	Baja: el HTML se mezcla con la lógica Java.
Mantenimiento	Difícil cuando la vista crece.
Vigencia	Es el cimiento de JSP y Spring; conviene conocerlo.

La principal molestia que muestra la Tabla 2.1 —HTML embebido en Java— motivó la creación de JSP, tema del Capítulo 3: invertir la relación y poner Java dentro del HTML.

Capítulo 3

JSP, EL y JSTL: vistas dinámicas

«Si el Servlet pone HTML dentro de Java, JSP pone Java dentro de HTML. La segunda forma es casi siempre más cómoda de leer.»

— Nota docente

Resultado de aprendizaje del capítulo

Al concluir el capítulo el estudiante construye vistas con JSP usando el *Expression Language* (EL) y la biblioteca JSTL, y separa responsabilidades aplicando el patrón Modelo–Vista–Controlador (MVC) con un Servlet como controlador y un JSP como vista.

En el Capítulo 2 el Servlet resolvió la lógica del lado del servidor, pero arrastraba un defecto incómodo: el HTML viajaba incrustado dentro del código Java, encerrado en cadenas y bloques de texto. Cada etiqueta, cada atributo y cada salto de línea de la página se escribía como una cadena más del programa. Para una vista real —con tablas, formularios y estilos— ese enfoque se vuelve ilegible, difícil de mantener y propenso a errores; además, obliga al programador de *backend* a editar el diseño y al diseñador a leer Java.

JavaServer Pages (JSP) invierte la relación. En lugar de escribir HTML dentro de Java, se escribe una página HTML que solo se “anima” en los puntos donde hacen falta datos dinámicos. El archivo se parece a una página web normal y el contenedor se encarga de convertirlo en un Servlet por debajo.

JavaServer Pages (JSP)

Un **JSP** es una página HTML que puede contener marcas especiales para insertar datos dinámicos. En la primera petición, el contenedor lo *traduce* automáticamente a un Servlet y lo compila; en las siguientes reutiliza esa clase. Es decir: un JSP *es* un Servlet, escrito al revés.

Sobre esa idea, la tecnología de vistas clásica de Java se apoya en tres piezas que este capítulo presenta una por una y luego integra:

- **JSP** aporta el *lienzo*: la página HTML que el contenedor traduce a un Servlet.
- **EL** (*Expression Language*) aporta el *acceso a los datos* con una sintaxis breve, sin escribir Java.
- **JSTL** (*JSP Standard Tag Library*) aporta la *lógica de presentación* —iterar, condicionar y formatear— mediante etiquetas.

JSP admite tres mecanismos para insertar dinamismo; la Tabla 3.1 los compara. La regla práctica del capítulo es simple: se prefieren EL y JSTL y se evitan los *scriptlets* (código Java suelto entre `<% y %>`), porque reintroducen la mezcla de lógica y presentación que justamente queremos eliminar.

Tabla 3.1: Mecanismos de dinamismo en JSP.

Mecanismo	Ejemplo	Recomendado
<i>Scriptlet</i>	<code><% int x = 5; %></code>	No (mezcla lógica)
Expresión EL	<code>\${usuario.nombre}</code>	Sí
Etiqueta JSTL	<code><c:forEach ...></code>	Sí

Como anticipa la Tabla 3.1, EL (`${...}`) lee los datos y JSTL (`<c:...>`) aporta las estructuras de control. El siguiente ejemplo introductorio, todavía sin entrar en el detalle de cada pieza, muestra el aspecto de un JSP: casi todo es HTML estático y solo unas pocas marcas cambian en cada petición.

Ejemplo introductorio — el aspecto de un JSP

Enunciado. Una página que saluda al visitante con el nombre recibido en la *query string* e informa qué método HTTP usó el navegador, sin necesidad de un controlador.

```

1 <%@ page contentType="text/html; charset=UTF-8" %>
2 <!DOCTYPE html>
3 <html lang="es-EC">
4 <head><meta charset="UTF-8"><title>Hola JSP</title></head>
5 <body>
6   <h1>Bienvenido a Aplicaciones Web</h1>
7   <p>Hola, ${param.nombre}.</p>                                <!-- EL lee ?nombre=...
   ↪ -->
8   <p>Metodo HTTP: ${pageContext.request.method}</p>
9 </body>
10 </html>

```

Listing 3.1: hola.jsp

Las marcas `${param.nombre}` y `${pageContext.request.method}` son expresiones EL que leen, respectivamente, un parámetro de la URL y un dato de la petición; el resto es HTML corriente. Las secciones siguientes desarrollan, en orden, cómo el contenedor convierte la página en un Servlet, cómo EL accede a los datos y cómo JSTL incorpora la lógica de presentación, para terminar integrando las tres piezas con el patrón MVC.

3.1 JSP: la página que se convierte en Servlet

Antes de usar EL y JSTL conviene entender qué ocurre por debajo cuando el contenedor recibe una petición a un archivo `.jsp`, porque eso explica tanto sus ventajas como sus limitaciones.

3.1.1 Del JSP al Servlet: el ciclo de traducción

La primera vez que se solicita un JSP, el contenedor (Tomcat) ejecuta una *traducción*: genera un archivo `.java` que es un Servlet equivalente, lo compila a `.class` y lo ejecuta para producir el HTML. En las peticiones siguientes reutiliza esa clase ya compilada, de modo que solo la primera carga paga el costo de traducir y compilar. La Figura 3.1 resume el proceso.



Figura 3.1: Ciclo de traducción de un JSP: el contenedor lo convierte en un Servlet `.java`, lo compila y lo ejecuta; las visitas posteriores reutilizan la clase ya compilada.

La consecuencia de la Figura 3.1 es doble. Por un lado, un JSP hereda todo el modelo del Servlet visto en el capítulo anterior: el mismo ciclo de vida, los mismos objetos `request` y `response` y la misma regla de concurrencia. Por otro, los errores de compilación de la página pueden aparecer en la primera visita —no al arrancar el servidor—, por lo que conviene probar cada vista al menos una vez.

3.1.2 Directivas: configurar la página

Una **directiva** es una instrucción para el traductor, no para el navegador; se escribe entre `<% y %>`. Las dos más frecuentes son `page`, que fija opciones como el tipo de contenido y la codificación, y `taglib`, que importa una biblioteca de etiquetas como JSTL. El Listado 3.2 muestra ambas en la cabecera de una página.

```
1 <%@ page contentType="text/html; charset=UTF-8" %>
2 <%@ taglib uri="jakarta.tags.core" prefix="c" %>
```

Listing 3.2: Directivas típicas en la cabecera de un JSP

Como indica el Listado 3.2, la directiva `taglib` asocia un prefijo (aquí `c`) a la biblioteca, de modo que después podremos escribir etiquetas como `<c:forEach>`. Con la página configurada, el siguiente paso es poblarla con datos: ese es el trabajo de EL.

3.2 EL: el *Expression Language*

El **Expression Language** es la forma recomendada de leer datos dentro de un JSP. Sustituye a los *scriptlets* por una sintaxis breve y segura que no requiere escribir Java en la página.

3.2.1 Sintaxis y acceso a los datos

Toda expresión EL se escribe entre `${ }`, y su resultado se imprime en el lugar donde aparece. El acceso a propiedades usa el punto: `${curso.nombre}` invoca internamente al método `getNombre()` del objeto `curso`. EL ofrece además *objetos implícitos* —como `param` (parámetros de la petición), `header` (cabeceras) o `pageContext`— y un conjunto de operadores resumidos en la Tabla 3.2.

Tabla 3.2: Operadores de uso frecuente en EL.

Categoría	Operadores y ejemplo
Acceso a propiedad	<code>.</code> y <code>[]</code> ; p.ej. <code>\${curso.nombre}</code>
Aritméticos	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>%</code>
Relacionales	<code>==</code> <code>!=</code> <code><</code> <code>></code> <code><=</code> <code>>=</code> (o <code>eq</code> <code>ne</code> <code>lt</code> <code>gt</code> <code>le</code> <code>ge</code>)
Lógicos	<code>&&</code> <code> </code> <code>!</code> (o <code>and</code> <code>or</code> <code>not</code>)
Vacío / nulo	<code>empty</code> ; p.ej. <code>\${empty param.nombre}</code>
Condicional	<code>cond ? a : b</code>

La Tabla 3.2 permite construir expresiones que deciden qué mostrar sin una sola línea de Java, como ilustra el ejemplo corto siguiente.

3.2.2 Ejemplo corto: leer parámetros con EL

Ejemplo 3.1 (RESUELTO) — decisiones con EL

Enunciado. A partir de la URL `?nombre=Ana&edad=20`, saludar al visitante, indicar si es mayor de edad y advertir cuando falte el nombre.

```

1 <%@ page contentType="text/html; charset=UTF-8" %>
2 <!DOCTYPE html>
3 <html lang="es-EC">
4 <head><meta charset="UTF-8"><title>EL</title></head>
5 <body>
6   <p>${empty param.nombre
7     ? 'Falta el parametro nombre'
8     : 'Hola, '.concat(param.nombre)}</p>
9   <p>Mayoría de edad: ${param.edad >= 18 ? 'si' : 'no'}</p>
10 </body>
11 </html>

```

Listing 3.3: edad.jsp

El operador `empty` detecta el parámetro ausente, el ternario elige el mensaje y el relacional `>=` compara la edad: toda la lógica de presentación cabe en dos expresiones. Para repetir o ramificar bloques completos, sin embargo, EL se queda corto; ahí entra JSTL.

3.3 JSTL: la biblioteca estándar de etiquetas

JSTL (*JSP Standard Tag Library*) aporta etiquetas para las tareas que EL no cubre: iterar colecciones, ramificar el HTML y formatear valores. Al ser etiquetas, se integran con naturalidad en el marcado de la página.

3.3.1 Etiquetas de iteración y condición

El núcleo (*core*, prefijo `c`) concentra las etiquetas más usadas. La Tabla 3.3 reúne las imprescindibles; todas requieren la directiva `taglib` de la cabecera.

Tabla 3.3: Etiquetas *core* de JSTL de uso frecuente.

Etiqueta	Propósito
<code><c:forEach></code>	Recorre una colección o un rango numérico.
<code><c:if></code>	Incluye el bloque si una condición es verdadera.
<code><c:choose></code>	Selección múltiple con <code><c:when></code> y <code><c:otherwise></code> .
<code><c:set></code>	Crea o asigna una variable.
<code><c:out></code>	Imprime un valor escapando caracteres HTML.

De las etiquetas de la Tabla 3.3, `<c:forEach>` es la más habitual al generar tablas y listas, y suele combinarse con `<c:if>` para casos especiales, como veremos a continuación.

3.3.2 Ejemplo corto: recorrer una lista con `c:forEach`

Ejemplo 3.2 (RESUELTO) — lista con JSTL

Enunciado. Mostrar una lista de lenguajes y, si la colección está vacía, un aviso. Para que el ejemplo sea autónomo, la lista se define en la propia página con `<c:set>` y un literal de EL.

```

1 <%@ page contentType="text/html; charset=UTF-8" %>
2 <%@ taglib uri="jakarta.tags.core" prefix="c" %>
3 <c:set var="lenguajes" value="${['Java','PHP','Python']}" />
4 <ul>
5   <c:forEach var="lng" items="${lenguajes}">
6     <li><c:out value="${lng}" /></li>
7   </c:forEach>
8 </ul>
9 <c:if test="${empty lenguajes}">
10   <p>No hay elementos para mostrar.</p>
11 </c:if>

```

Listing 3.4: lenguajes.jsp

La etiqueta `<c:forEach>` repite el bloque `<li>` por cada elemento, `<c:out>` lo imprime de forma segura y `<c:if>` cubre el caso de lista vacía. Con JSP, EL y JSTL ya presentados por separado, el cierre del capítulo los integra en una aplicación con la arquitectura correcta.

3.4 Ejemplo completo: listado de cursos con MVC

Los ejemplos anteriores eran páginas aisladas. Una aplicación real separa la lógica de la presentación con el patrón **MVC** (Modelo–Vista–Controlador), en el que el Servlet coordina y el JSP solo dibuja. Esta sección lo construye en tres pasos: el diseño del patrón, su implementación en tres archivos y la ejecución del resultado.

3.4.1 Diseño: el patrón MVC en tres roles

Mezclar lógica y presentación es el principal defecto del Servlet puro. El patrón MVC lo corrige separando tres roles, que muestra la Figura 3.2: el **Modelo** (datos), la **Vista** (JSP que dibuja) y el **Controlador** (Servlet que coordina).

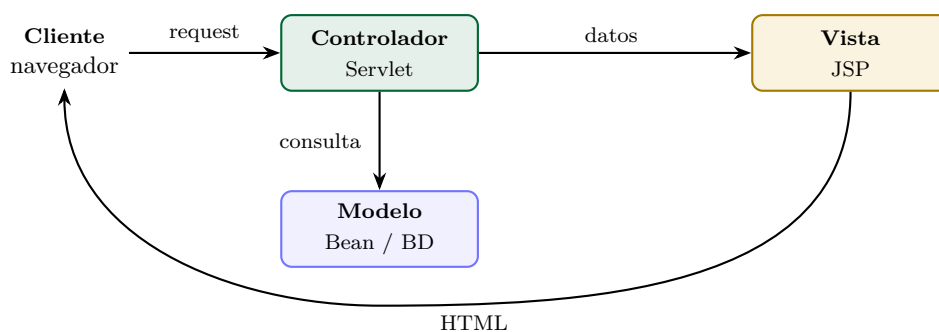


Figura 3.2: Patrón MVC: el controlador (Servlet) obtiene los datos del modelo y los entrega a la vista (JSP), que produce el HTML para el cliente.

El flujo de la Figura 3.2 se traduce en tres archivos —uno por rol— que implementamos a continuación.

3.4.2 Implementación: modelo, controlador y vista

Ejemplo 3.3 (RESUELTO) — listado de cursos con MVC

Enunciado. Mostrar una tabla de cursos. El **Modelo** es una clase `Curso`; el **Controlador** es un Servlet que prepara la lista; la **Vista** es un JSP que la dibuja con EL y JSTL.

1) Modelo — `Curso.java` (un *record* de Java 21):

```

1 package ec.edu.uteq.unidad2.model;
2
3 public record Curso(int id, String nombre, int credits) { }
```

Listing 3.5: `Curso.java`

2) Controlador — `CursoServlet.java`:

```

1 package ec.edu.uteq.unidad2.controller;
2
3 import ec.edu.uteq.unidad2.model.Curso;
4 import jakarta.servlet.ServletException;
5 import jakarta.servlet.annotation.WebServlet;
```

```

6 import jakarta.servlet.http.*;
7 import java.io.IOException;
8 import java.util.List;
9
10 @WebServlet("/cursos")
11 public class CursoServlet extends HttpServlet {
12     @Override
13     protected void doGet(HttpServletRequest req, HttpServletResponse
        ↪ resp)
14         throws ServletException, IOException {
15         List<Curso> cursos = List.of(           // modelo (datos)
16             new Curso(1, "Aplicaciones Web", 3),
17             new Curso(2, "Base de Datos", 4),
18             new Curso(3, "Estructura de Datos", 4));
19         req.setAttribute("cursos", cursos);      // entrega a la
        ↪ vista
20         req.getRequestDispatcher("/WEB-INF/cursos.jsp")
21             .forward(req, resp);                 // delega el
        ↪ render al JSP
22     }
23 }

```

Listing 3.6: CursoServlet.java

3) Vista — cursos.jsp (EL + JSTL):

```

1 <%@ taglib uri="jakarta.tags.core" prefix="c" %>
2 <!DOCTYPE html>
3 <html lang="es-EC">
4 <head><meta charset="UTF-8"><title>Cursos</title></head>
5 <body>
6     <h1>Oferta de cursos</h1>
7     <table border="1" cellpadding="6">
8         <tr><th>ID</th><th>Nombre</th><th>Créditos</th></tr>
9         <c:forEach var="curso" items="${cursos}">
10             <tr>
11                 <td>${curso.id}</td>
12                 <td>${curso.nombre}</td>
13                 <td>${curso.creditos}</td>
14             </tr>
15         </c:forEach>
16     </table>
17 </body>
18 </html>

```

Listing 3.7: WEB-INF/cursos.jsp

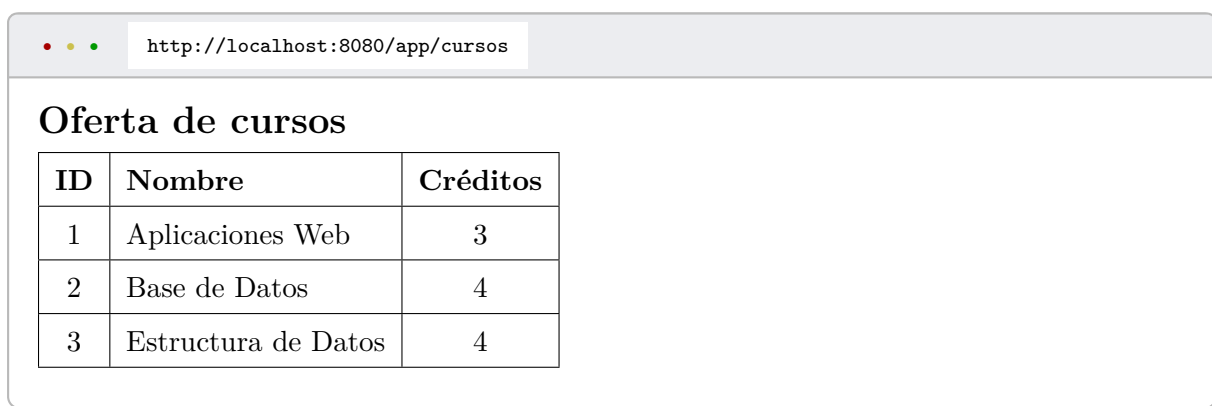
Por qué la vista vive en WEB-INF

Los archivos bajo `/WEB-INF` no son accesibles directamente por URL. Al colocar el JSP allí, el usuario *no* puede abrir `cursos.jsp` sin pasar por el controlador: se garantiza que la vista siempre reciba sus datos.

En la vista, EL (`${curso.nombre}`) lee cada propiedad y JSTL (`<c:forEach>`) recorre la lista que preparó el controlador: las dos piezas estudiadas antes trabajan juntas sobre el JSP.

3.4.3 Ejecución y resultado

Al visitar `/cursos`, el controlador construye la lista y reenvía la petición al JSP, que genera la tabla. La Figura 3.3 reproduce la pantalla obtenida.



ID	Nombre	Créditos
1	Aplicaciones Web	3
2	Base de Datos	4
3	Estructura de Datos	4

Figura 3.3: Ejecución del Ejemplo 3.3: el JSP recorrió con `c:forEach` la lista preparada por el controlador y generó la tabla HTML.

La Figura 3.3 demuestra la ventaja del MVC: la lógica vive en el Servlet y la presentación en el JSP, sin mezclarse. Aun así, configurar Servlets y JSP a mano sigue siendo laborioso. El Capítulo 4 introduce Spring Boot, que automatiza casi toda esa fontanería.

Capítulo 4

Spring Boot: el estándar moderno

«La mejor configuración es la que no hay que escribir.»

— Filosofía de *convención sobre configuración* de Spring Boot

Resultado de aprendizaje del capítulo

Al concluir el capítulo el estudiante crea un proyecto Spring Boot, explica la inyección de dependencias y la autoconfiguración, y expone un primer endpoint que devuelve JSON, comprobando su ejecución en el navegador.

4.1 ¿Qué resuelve Spring Boot?

Los Capítulos 2 y 3 mostraron que escribir HTML en Java es incómodo y que configurar Servlets a mano es laborioso. Spring Boot elimina casi toda esa fricción. Lo veremos en dos planos: las tres ideas que sostienen el framework y, dentro de ellas, la que más impacta el día a día, la inyección de dependencias.

4.1.1 Las tres ideas centrales

Spring Boot descansa sobre tres ideas, recogidas en la Tabla 4.1.

Tabla 4.1: Las tres ideas centrales de Spring Boot.

Idea	Qué significa
Inversión de control (IoC)	El framework crea y conecta los objetos por ti.
Autoconfiguración	Detecta las librerías presentes y configura lo razonable.
<i>Starters</i>	Dependencias preempaquetadas (<code>spring-boot-starter-web</code> , ...).

4.1.2 Inyección de dependencias en una frase

Inyección de dependencias (DI)

En lugar de que una clase cree los objetos que necesita con `new`, los *recibe* ya contruidos. Spring mantiene un *contenedor* de objetos (*beans*) y los entrega donde hagan falta. El beneficio: clases desacopladas, fáciles de probar y de sustituir [5].

La diferencia es visible en el código. Sin DI:

```
1 public class CursoServicio {
2     private final CursoRepo repo = new CursoRepo(); // crea su dependencia
3 }
```

Listing 4.1: Acoplamiento rígido (a evitar)

Con DI (Spring inyecta el *bean* por el constructor):

```
1 @Service
2 public class CursoServicio {
3     private final CursoRepo repo;
4     public CursoServicio(CursoRepo repo) { // Spring lo provee
5         this.repo = repo;
6     }
7 }
```

Listing 4.2: Desacoplado con inyección por constructor

4.2 Estructura de un proyecto Spring Boot

Un proyecto Spring Boot tiene una estructura convencional. La Figura 4.1 la presenta como árbol de directorios.

```
tienda/
  pom.xml (dependencias Maven)
  src/main/java/ec/edu/uteq/tienda/
    TiendaApplication.java (punto de arranque)
    controller/  service/  repository/  model/
  src/main/resources/
    application.properties (configuración)
    templates/ (vistas Thymeleaf)
    static/ (CSS, JS, imágenes)
```

Figura 4.1: Estructura convencional de un proyecto Spring Boot con Maven.

El archivo `pom.xml` de la Figura 4.1 declara las dependencias. Para una aplicación web mínima basta el *starter* web:

```
1 <dependencies>
2   <dependency>
```

```

3     <groupId>org.springframework.boot</groupId>
4     <artifactId>spring-boot-starter-web</artifactId>
5 </dependency>
6 <dependency>
7     <groupId>org.springframework.boot</groupId>
8     <artifactId>spring-boot-starter-thymeleaf</artifactId>
9 </dependency>
10 </dependencies>

```

Listing 4.3: Fragmento de pom.xml

4.3 Ejemplo corto: primer endpoint REST

Ejemplo 4.1 (RESUELTO) — “Hola” en JSON

Enunciado. Crear una aplicación Spring Boot con un controlador REST que devuelva un objeto JSON con un saludo.

Paso 1. En IntelliJ IDEA Ultimate: File → New → Project → Spring Boot. Elegir Java 21, Maven, Spring Boot 3.4 y la dependencia *Spring Web*.

Paso 2. Clase de arranque (la genera el asistente):

```

1 package ec.edu.uteq.tienda;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5
6 @SpringBootApplication                                // habilita
   ↳ autoconfiguración
7 public class TiendaApplication {
8     public static void main(String[] args) {
9         SpringApplication.run(TiendaApplication.class, args);
10    }
11 }

```

Listing 4.4: TiendaApplication.java

Paso 3. Controlador REST:

```

1 package ec.edu.uteq.tienda.controller;
2
3 import org.springframework.web.bind.annotation.*;
4 import java.util.Map;
5
6 @RestController                                    // cada método devuelve
   ↳ JSON
7 @RequestMapping("/api")
8 public class SaludoController {
9
10     @GetMapping("/saludo")                            // GET /api/saludo
11     public Map<String, Object> saludo(
12         @RequestParam(defaultValue = "Mundo") String nombre) {

```

```
13         return Map.of(  
14             "mensaje", "Hola, " + nombre,  
15             "framework", "Spring Boot 3.4");  
16     }  
17 }
```

Listing 4.5: SaludoController.java

Paso 4. Pulsar el botón verde de ejecución (**Shift+F10**). En la consola de IntelliJ aparece el banner de Spring Boot y, tras unos segundos, el mensaje de que Tomcat escucha en el puerto 8080. La Figura 4.2 reproduce esa salida.

```
. ____ _
/\V ___'___ _(_)_____ \\\
( )\___ | '_| |'_||'_ \V_' | \\\
\V/ ___)|_|_)| | | | | |_(| |) ) )
'|____| .__|_|_|_| |_|_, | / / /
:: Spring Boot :: (v3.4.0)

INFO -- Starting TiendaApplication using Java 21
INFO -- Tomcat initialized with port 8080 (http)
INFO -- Started TiendaApplication in 1.842 seconds
INFO -- Tomcat started on port 8080 (http)
```

Figura 4.2: Ejecución del Ejemplo 4.1: la consola de IntelliJ muestra el arranque de Spring Boot y Tomcat embebido en el puerto 8080.

Paso 5. Abrir el navegador en la URL del endpoint. El servidor responde con JSON, que el navegador muestra tal cual (Figura 4.3).

```
http://localhost:8080/api/saludo?nombre=UTEQ

{
  "mensaje": "Hola, UTEQ",
  "framework": "Spring Boot 3.4"
}
```

Figura 4.3: Ejecución del Ejemplo 4.1: el endpoint devuelve un objeto JSON; Spring serializó automáticamente el **Map** de Java.

Comparando el Ejemplo 4.1 con el Servlet del Capítulo 2 se aprecia la productividad de Spring Boot (Figuras 4.2 y 4.3): no hubo que registrar servlets, configurar XML ni serializar JSON a mano. El Capítulo 5 aprovecha esta base para construir una aplicación completa con interfaz de usuario y base de datos.

4.4 Configuración con `application.properties`

Spring Boot lee su configuración de `src/main/resources/application.properties` (o `.yml`).

Un ejemplo típico para la aplicación que sigue:

```
1 # Puerto del servidor embebido
2 server.port=8080
3 # Nombre visible de la aplicacion
4 spring.application.name=tienda-uteq
5 # Mostrar las sentencias SQL en consola (util en desarrollo)
6 spring.jpa.show-sql=true
```

Listing 4.6: `application.properties`

Capítulo 5

Aplicación completa: CRUD con frontend y backend

«Una aplicación no se entiende leyendo capas sueltas, sino siguiendo el viaje de un dato desde el formulario hasta la base y de vuelta.»

— Nota docente

Resultado de aprendizaje del capítulo

Al concluir el capítulo el estudiante construye una aplicación web completa con Spring Boot que crea, lee, actualiza y elimina registros (CRUD), con persistencia real en base de datos, una interfaz de usuario en Thymeleaf y validación integral, comprobando cada operación en el navegador.

5.1 Qué vamos a construir

Construiremos **TiendaUTEQ**, un catálogo de productos con las cuatro operaciones CRUD. La Tabla 5.1 relaciona cada operación con su verbo HTTP y su ruta.

Tabla 5.1: Operaciones CRUD de la aplicación TiendaUTEQ.

Operación	Verbo	Ruta	Vista
Listar (Read)	GET	/productos	tabla
Nuevo (Create)	GET	/productos/nuevo	formulario vacío
Guardar	POST	/productos	redirige a la tabla
Editar (Update)	GET	/productos/{id}/editar	formulario lleno
Eliminar (Delete)	POST	/productos/{id}/eliminar	redirige a la tabla

La aplicación se organiza en cuatro capas, según la Figura 5.1: la petición entra por el **controlador**, que llama al **servicio** (reglas de negocio), que usa el **repositorio** (acceso a datos), que habla con la **base de datos**.

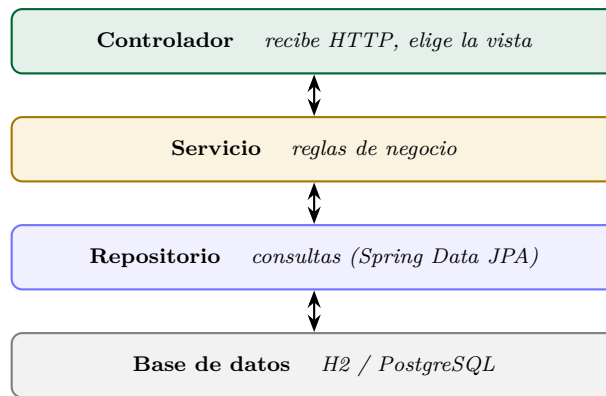


Figura 5.1: Arquitectura por capas de TiendaUTEQ. Cada capa solo conoce a la inmediatamente inferior, lo que facilita pruebas y mantenimiento.

5.2 Backend

5.2.1 Dependencias y configuración

Al *starter* web añadimos JPA, la base H2 (embebida, sin instalación) y la validación. El `pom.xml` declara:

```

1 <dependency>
2   <groupId>org.springframework.boot</groupId>
3   <artifactId>spring-boot-starter-web</artifactId>
4 </dependency>
5 <dependency>
6   <groupId>org.springframework.boot</groupId>
7   <artifactId>spring-boot-starter-thymeleaf</artifactId>
8 </dependency>
9 <dependency>
10  <groupId>org.springframework.boot</groupId>
11  <artifactId>spring-boot-starter-data-jpa</artifactId>
12 </dependency>
13 <dependency>
14  <groupId>org.springframework.boot</groupId>
15  <artifactId>spring-boot-starter-validation</artifactId>
16 </dependency>
17 <dependency>
18  <groupId>com.h2database</groupId>
19  <artifactId>h2</artifactId>
20  <scope>runtime</scope>
21 </dependency>

```

Listing 5.1: Dependencias en `pom.xml`

La configuración activa la base H2 en memoria y su consola web:

```

1 spring.datasource.url=jdbc:h2:mem:tienda
2 spring.datasource.driver-class-name=org.h2.Driver
3 spring.jpa.hibernate.ddl-auto=update

```

```
4 spring.h2.console.enabled=true
5 spring.jpa.show-sql=true
```

Listing 5.2: application.properties

5.2.2 Modelo: la entidad Producto

La entidad es una clase Java mapeada a una tabla. Las anotaciones de `jakarta.validation` declaran las reglas que se aplicarán al guardar [3].

```
1 package ec.edu.uteq.tienda.model;
2
3 import jakarta.persistence.*;
4 import jakarta.validation.constraints.*;
5 import java.math.BigDecimal;
6
7 @Entity
8 @Table(name = "productos")
9 public class Producto {
10
11     @Id
12     @GeneratedValue(strategy = GenerationType.IDENTITY)
13     private Long id;
14
15     @NotBlank(message = "El nombre es obligatorio")
16     @Size(min = 3, max = 100, message = "Entre 3 y 100 caracteres")
17     private String nombre;
18
19     @NotNull(message = "El precio es obligatorio")
20     @DecimalMin(value = "0.01", message = "El precio debe ser mayor que 0")
21     private BigDecimal precio;
22
23     @NotNull(message = "El stock es obligatorio")
24     @Min(value = 0, message = "El stock no puede ser negativo")
25     private Integer stock;
26
27     public Producto() { }           // constructor requerido por JPA
28
29     // Getters y setters
30     public Long getId()             { return id; }
31     public void setId(Long id)      { this.id = id; }
32     public String getNombre()       { return nombre; }
33     public void setNombre(String n) { this.nombre = n; }
34     public BigDecimal getPrecio()   { return precio; }
35     public void setPrecio(BigDecimal p) { this.precio = p; }
36     public Integer getStock()       { return stock; }
37     public void setStock(Integer s) { this.stock = s; }
38 }
```

Listing 5.3: model/Producto.java

5.2.3 Repositorio

Spring Data JPA genera la implementación del CRUD a partir de una interfaz: no se escribe SQL para las operaciones básicas. Basta con extender `JpaRepository`.

```
1 package ec.edu.uteq.tienda.repository;
2
3 import ec.edu.uteq.tienda.model.Producto;
4 import org.springframework.data.jpa.repository.JpaRepository;
5 import java.util.List;
6
7 public interface ProductoRepository
8     extends JpaRepository<Producto, Long> {
9
10     // Consulta derivada: Spring la implementa por el nombre del metodo
11     List<Producto> findByNombreContainingIgnoreCase(String texto);
12 }
```

Listing 5.4: repository/ProductoRepository.java

5.2.4 Servicio

El servicio concentra las reglas de negocio y aísla al controlador del repositorio. Aquí añadimos una regla simple: no permitir nombres duplicados.

```
1 package ec.edu.uteq.tienda.service;
2
3 import ec.edu.uteq.tienda.model.Producto;
4 import ec.edu.uteq.tienda.repository.ProductoRepository;
5 import org.springframework.stereotype.Service;
6 import java.util.List;
7
8 @Service
9 public class ProductoService {
10
11     private final ProductoRepository repo;
12
13     public ProductoService(ProductoRepository repo) { // inyección por
14         ↪ constructor
15         this.repo = repo;
16     }
17
18     public List<Producto> listar() { return repo.findAll(); }
19     public Producto buscar(Long id) { return repo.findById(id).
20         ↪ orElseThrow(); }
21     public void guardar(Producto p) { repo.save(p); }
22     public void eliminar(Long id) { repo.deleteById(id); }
23     public List<Producto> buscarPorNombre(String t) {
24         return repo.findByNombreContainingIgnoreCase(t);
25     }
26 }
```

Listing 5.5: service/ProductoService.java

5.2.5 Controlador web

El controlador mapea cada ruta de la Tabla 5.1 a un método, prepara el modelo y elige la vista. La anotación `@Valid` dispara la validación de la entidad; `BindingResult` recoge los errores.

```

1 package ec.edu.uteq.tienda.controller;
2
3 import ec.edu.uteq.tienda.model.Producto;
4 import ec.edu.uteq.tienda.service.ProductoService;
5 import jakarta.validation.Valid;
6 import org.springframework.stereotype.Controller;
7 import org.springframework.ui.Model;
8 import org.springframework.validation.BindingResult;
9 import org.springframework.web.bind.annotation.*;
10 import org.springframework.web.servlet.mvc.support.RedirectAttributes;
11
12 @Controller
13 @RequestMapping("/productos")
14 public class ProductoController {
15
16     private final ProductoService servicio;
17
18     public ProductoController(ProductoService servicio) {
19         this.servicio = servicio;
20     }
21
22     @GetMapping                                // GET /productos
23     public String listar(Model model) {
24         model.addAttribute("productos", servicio.listar());
25         return "productos/lista";              // -> templates/productos/
26         ↪ lista.html
27     }
28
29     @GetMapping("/nuevo")                       // formulario vacío
30     public String nuevo(Model model) {
31         model.addAttribute("producto", new Producto());
32         return "productos/form";
33     }
34
35     @GetMapping("/{id}/editar")                 // formulario lleno
36     public String editar(@PathVariable Long id, Model model) {
37         model.addAttribute("producto", servicio.buscar(id));
38         return "productos/form";
39     }
40
41     @PostMapping                               // crear o actualizar
42     public String guardar(@Valid @ModelAttribute("producto") Producto
43         ↪ producto,
44                             BindingResult result,
45                             RedirectAttributes flash) {
46         if (result.hasErrors()) {               // hay errores de validaci
47             ↪ ón
48             return "productos/form";           // remuestra el

```

```

46         ↪ formulario
47     }
48     servicio.guardar(producto);
49     flash.addFlashAttribute("ok", "Producto guardado correctamente.");
50     return "redirect:/productos";           // patrón POST-Redirect-
51     ↪ GET
52 }
53
54 @PostMapping("/{id}/eliminar")
55 public String eliminar(@PathVariable Long id, RedirectAttributes flash) {
56     servicio.eliminar(id);
57     flash.addFlashAttribute("ok", "Producto eliminado.");
58     return "redirect:/productos";
59 }
60 }

```

Listing 5.6: controller/ProductoController.java

Patrón POST–Redirect–GET

Tras un POST exitoso, el controlador no devuelve HTML sino una **redirección** (`redirect:/productos`). Así, si el usuario recarga la página, no reenvía el formulario por accidente. El mensaje de éxito viaja como *flash attribute*: vive solo hasta la siguiente petición.

5.2.6 Datos de prueba

Para no arrancar con la tabla vacía, un `CommandLineRunner` inserta tres productos al iniciar:

```

1 package ec.edu.uteq.tienda.config;
2
3 import ec.edu.uteq.tienda.model.Producto;
4 import ec.edu.uteq.tienda.repository.ProductoRepository;
5 import org.springframework.boot.CommandLineRunner;
6 import org.springframework.context.annotation.Bean;
7 import org.springframework.context.annotation.Configuration;
8 import java.math.BigDecimal;
9
10 @Configuration
11 public class DatosIniciales {
12     @Bean
13     CommandLineRunner cargar(ProductRepository repo) {
14         return args -> {
15             repo.save(crear("Teclado mecánico", "45.90", 30));
16             repo.save(crear("Mouse inalámbrico", "18.50", 75));
17             repo.save(crear("Monitor 24\"", "139.00", 12));
18         };
19     }
20     private Producto crear(String n, String p, int s) {
21         Producto x = new Producto();
22         x.setNombre(n); x.setPrecio(new BigDecimal(p)); x.setStock(s);
23         return x;
24     }
25 }

```

```

24     }
25 }

```

Listing 5.7: config/DatosIniciales.java

5.3 Frontend

Las vistas usan **Thymeleaf**, un motor de plantillas que produce HTML válido y legible incluso sin servidor. Para el estilo incorporamos *Bootstrap 5* por CDN. Todas las plantillas viven en `src/main/resources/templates/`.

5.3.1 Vista de listado

La plantilla recorre los productos con `th:each` y construye la tabla. Muestra el mensaje de éxito si existe y enlaza a editar y eliminar.

```

1 <!DOCTYPE html>
2 <html lang="es-EC" xmlns:th="http://www.thymeleaf.org">
3 <head>
4   <meta charset="UTF-8"><title>Productos</title>
5   <link rel="stylesheet"
6     href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap
7     ↪ .min.css">
8 </head>
9 <body class="container py-4">
10   <h1 class="mb-3">Catálogo de productos</h1>
11   <div th:if="${ok}" class="alert alert-success" th:text="${ok}"></div>
12   <a th:href="@{/productos/nuevo}" class="btn btn-success mb-3">+ Nuevo
13     ↪ producto</a>
14   <table class="table table-striped table-bordered">
15     <thead class="table-dark">
16       <tr><th>ID</th><th>Nombre</th><th>Precio</th><th>Stock</th><th>Acciones
17         ↪ </th></tr>
18     </thead>
19     <tbody>
20       <tr th:each="p : ${productos}">
21         <td th:text="${p.id}">1</td>
22         <td th:text="${p.nombre}">Producto</td>
23         <td th:text="'$' + ${p.precio}">$0.00</td>
24         <td th:text="${p.stock}">0</td>
25         <td>
26           <a th:href="@{/productos/{id}/editar(id=${p.id})}"
27             class="btn btn-sm btn-primary">Editar</a>
28           <form th:action="@{/productos/{id}/eliminar(id=${p.id})}"
29             method="post" style="display:inline"
30             onsubmit="return confirm('¿Eliminar este producto?');">
31             <button class="btn btn-sm btn-danger">Eliminar</button>
32           </form>

```

```

33         </td>
34     </tr>
35 </tbody>
36 </table>
37 </body>
38 </html>

```

Listing 5.8: templates/productos/lista.html

5.3.2 Vista de formulario

El mismo formulario sirve para crear y editar (Spring distingue por el id oculto). El atributo `th:errors` muestra los mensajes de validación junto a cada campo.

```

1 <!DOCTYPE html>
2 <html lang="es-EC" xmlns:th="http://www.thymeleaf.org">
3 <head>
4     <meta charset="UTF-8"><title>Producto</title>
5     <link rel="stylesheet"
6         href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap
7             ↪ .min.css">
8 </head>
9 <body class="container py-4" style="max-width:520px">
10     <h1 th:text="${producto.id} == null ? 'Nuevo producto' : 'Editar producto'"
11         ↪ >
12         Producto</h1>
13
14
15     <form th:action="@{/productos}" th:object="${producto}" method="post">
16         <input type="hidden" th:field="*{id}">
17
18         <div class="mb-3">
19             <label class="form-label">Nombre</label>
20             <input class="form-control" th:field="*{nombre}">
21             <div class="text-danger" th:if="${#fields.hasErrors('nombre')}"
22                 th:errors="*{nombre}"></div>
23         </div>
24
25         <div class="mb-3">
26             <label class="form-label">Precio (USD)</label>
27             <input class="form-control" type="number" step="0.01" th:field="*{
28                 ↪ precio}">
29             <div class="text-danger" th:if="${#fields.hasErrors('precio')}"
30                 th:errors="*{precio}"></div>
31         </div>
32
33         <div class="mb-3">
34             <label class="form-label">Stock</label>
35             <input class="form-control" type="number" th:field="*{stock}">
36             <div class="text-danger" th:if="${#fields.hasErrors('stock')}"
37                 th:errors="*{stock}"></div>
38         </div>

```

```

36     <button class="btn btn-success">Guardar</button>
37     <a th:href="@{/productos}" class="btn btn-secondary">Cancelar</a>
38 </form>
39 </body>
40 </html>

```

Listing 5.9: templates/productos/form.html

5.4 Ejecución

Con todo en su lugar, ejecutamos la aplicación (**Shift+F10**) y abrimos `/productos`. La Figura 5.2 muestra el catálogo con los tres productos de prueba.

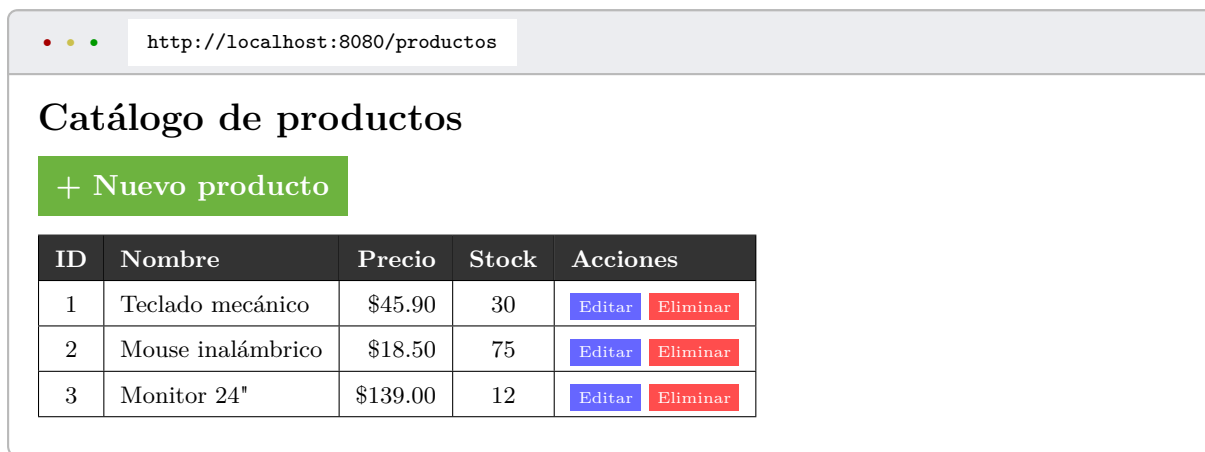
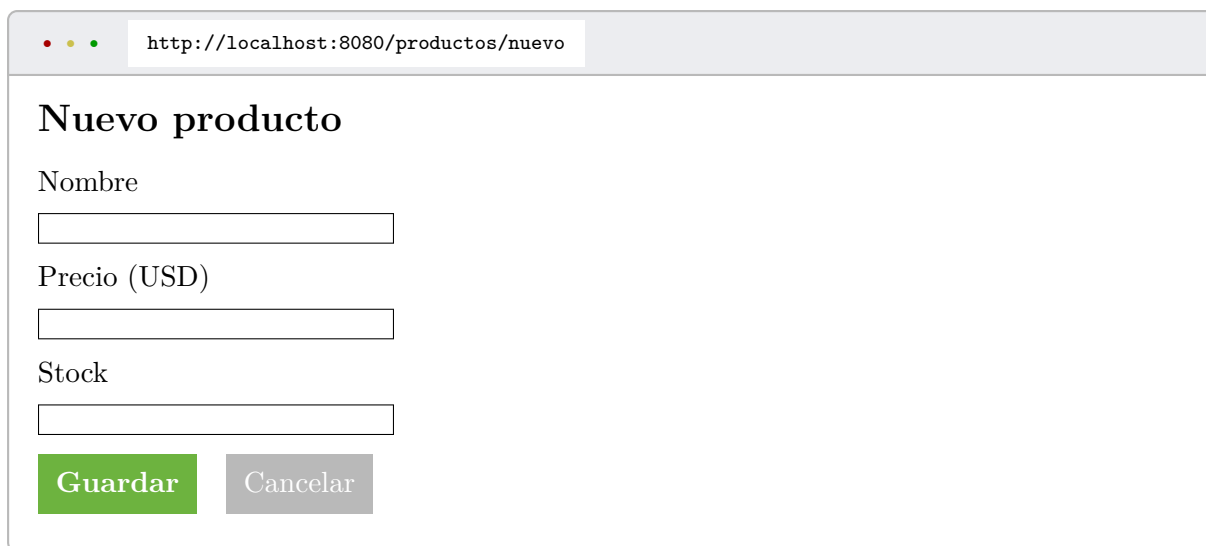


Figura 5.2: Ejecución de TiendaUTeQ: vista de listado (**Read**) con los datos de prueba y los botones de acción.

Al pulsar «+ Nuevo producto» se abre el formulario vacío (Figura 5.3). Si se envía con datos inválidos —por ejemplo, precio 0 y nombre demasiado corto— la validación del servidor rechaza el envío y muestra los mensajes (Figura 5.4).



http://localhost:8080/productos/nuevo

Nuevo producto

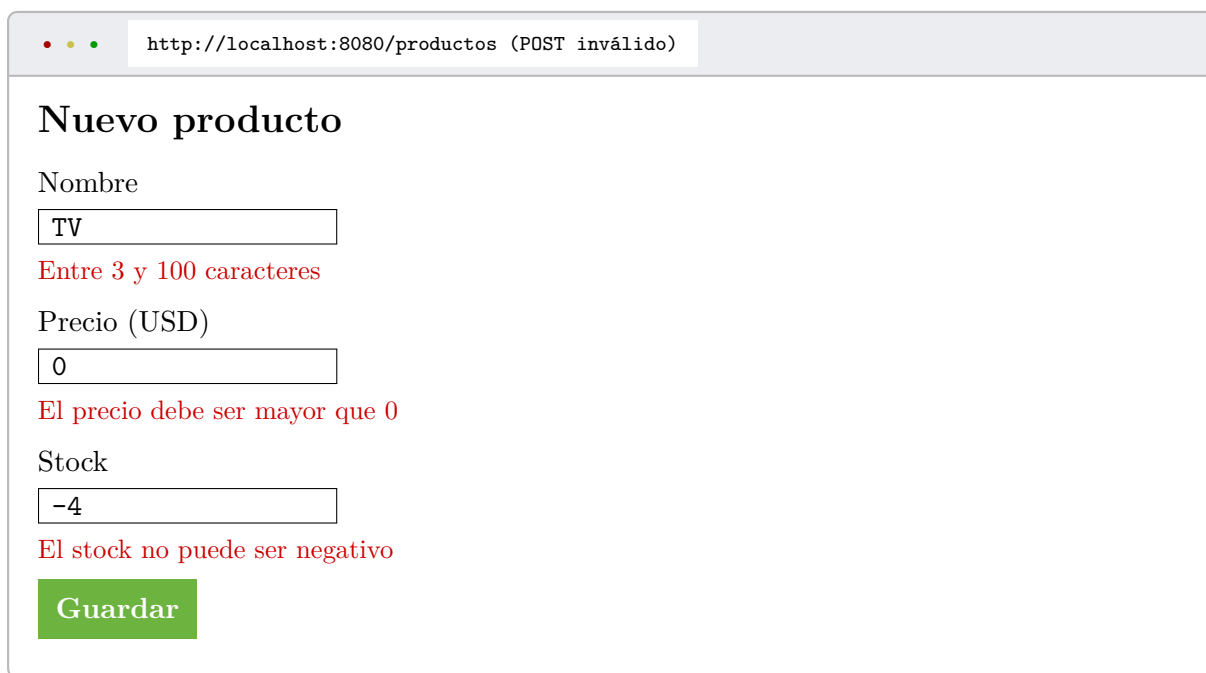
Nombre

Precio (USD)

Stock

Guardar Cancelar

Figura 5.3: Ejecución de TiendaUTEQ: formulario de alta (Create).



http://localhost:8080/productos (POST inválido)

Nuevo producto

Nombre

Entre 3 y 100 caracteres

Precio (USD)

El precio debe ser mayor que 0

Stock

El stock no puede ser negativo

Guardar

Figura 5.4: Ejecución de TiendaUTEQ: la validación del servidor (@Valid) rechaza los datos inválidos y muestra un mensaje por cada campo.

Cuando los datos son válidos, el producto se guarda, la aplicación redirige al listado y muestra el mensaje de éxito en verde (Figura 5.5). La secuencia completa de las Figuras 5.2–5.5 cubre las cuatro operaciones de la Tabla 5.1.



Figura 5.5: Ejecución de TiendaUTEQ: tras un guardado válido, el patrón POST-Redirect-GET lleva al listado con el mensaje *flash* de éxito.

5.5 Inspección de la base de datos

Como activamos la consola de H2, podemos verificar que los datos se persistieron realmente abriendo `/h2-console` y ejecutando una consulta SQL. La Figura 5.6 reproduce el resultado.

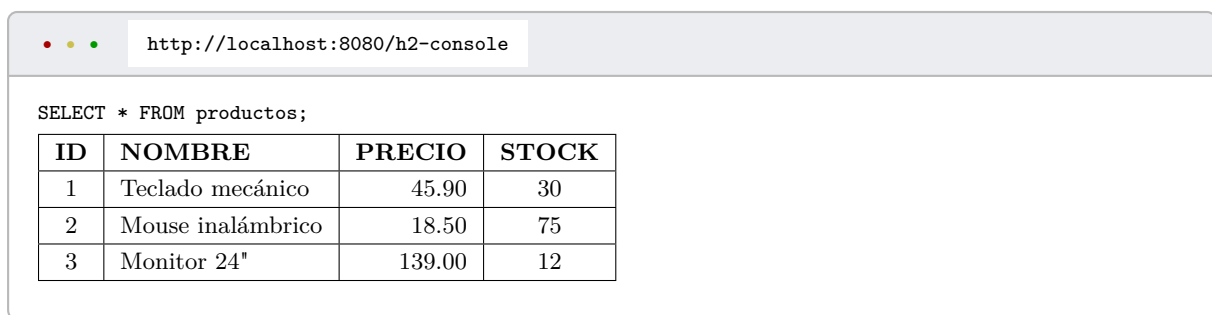


Figura 5.6: Verificación de la persistencia: la consola de H2 confirma que los registros se guardaron en la tabla `productos`.

La Figura 5.6 cierra el viaje del dato anunciado al inicio del capítulo: del formulario, a través de las capas de la Figura 5.1, hasta la tabla, y de regreso a la pantalla.

Capítulo 6

API REST y consumo desde el cliente

«El estilo REST no es una tecnología, es un conjunto de restricciones que, respetadas, producen sistemas web escalables.»

— Adaptado de Roy Fielding

Resultado de aprendizaje del capítulo

Al concluir el capítulo el estudiante expone los datos como una API REST en JSON con los códigos HTTP correctos y la consume desde el navegador con `fetch`, integrando el lado del servidor (Unidad II) con el lado del cliente (Unidad I).

6.1 De vistas a datos: ¿qué cambia?

El paso de la aplicación MVC del Capítulo 5 a una API REST cambia qué entrega el servidor y quién arma la interfaz. Lo abordamos en dos planos: el contraste conceptual entre devolver HTML y devolver JSON, y las convenciones de verbos y códigos de estado que formalizan ese contrato.

6.1.1 HTML armado frente a datos JSON

En el Capítulo 5 el servidor devolvía **HTML ya armado** (Thymeleaf). Una API REST devuelve **datos en JSON** y deja que el cliente decida cómo mostrarlos. La Figura 6.1 contrasta ambos enfoques.

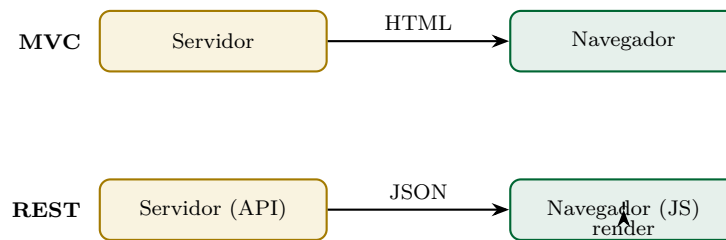


Figura 6.1: MVC envía HTML terminado; REST envía datos en JSON que el cliente transforma en interfaz.

Como muestra la Figura 6.1, REST es la base de las aplicaciones de página única (SPA) y de las apps móviles, que comparten el mismo backend.

6.1.2 Verbos HTTP y códigos de estado

REST asigna a cada operación un verbo y a cada resultado un código de estado. La Tabla 6.1 resume las convenciones que aplicaremos [4].

Tabla 6.1: Verbos HTTP y códigos de estado en una API REST de productos.

Verbo	Ruta	Acción	Éxito
GET	/api/productos	Listar todos	200 OK
GET	/api/productos/{id}	Obtener uno	200 / 404
POST	/api/productos	Crear	201 Created
PUT	/api/productos/{id}	Actualizar	200 OK
DELETE	/api/productos/{id}	Eliminar	204 No Content

6.2 Backend: el controlador REST

El controlador REST reutiliza el mismo servicio del Capítulo 5; solo cambia que devuelve objetos (serializados a JSON) y códigos HTTP mediante `ResponseEntity`, según la Tabla 6.1.

```

1 package ec.edu.uteq.tienda.api;
2
3 import ec.edu.uteq.tienda.model.Producto;
4 import ec.edu.uteq.tienda.service.ProductoService;
5 import jakarta.validation.Valid;
6 import org.springframework.http.*;
7 import org.springframework.web.bind.annotation.*;
8 import java.net.URI;
9 import java.util.List;
10
11 @RestController                                     // devuelve JSON, no vistas
12 @RequestMapping("/api/productos")
13 @CrossOrigin                                       // permite llamadas desde
14     ↪ el cliente
15 public class ProductoRestController {
16     private final ProductoService servicio;

```

```
17 public ProductoRestController(ProductoService s) { this.servicio = s; }
18
19 @GetMapping // 200 OK + lista
20 public List<Producto> listar() {
21     return servicio.listar();
22 }
23
24 @GetMapping("/{id}") // 200 OK o 404
25 public ResponseEntity<Producto> obtener(@PathVariable Long id) {
26     try {
27         return ResponseEntity.ok(servicio.buscar(id));
28     } catch (Exception e) {
29         return ResponseEntity.notFound().build();
30     }
31 }
32
33 @PostMapping // 201 Created + Location
34 public ResponseEntity<Producto> crear(@Valid @RequestBody Producto p) {
35     servicio.guardar(p);
36     URI ubic = URI.create("/api/productos/" + p.getId());
37     return ResponseEntity.created(ubic).body(p);
38 }
39
40 @PutMapping("/{id}") // 200 OK
41 public ResponseEntity<Producto> actualizar(@PathVariable Long id,
42                                           @Valid @RequestBody Producto
43                                           ⇨ p) {
44     p.setId(id);
45     servicio.guardar(p);
46     return ResponseEntity.ok(p);
47 }
48
49 @DeleteMapping("/{id}") // 204 No Content
50 public ResponseEntity<Void> eliminar(@PathVariable Long id) {
51     servicio.eliminar(id);
52     return ResponseEntity.noContent().build();
53 }
```

Listing 6.1: controller/ProductoRestController.java

Al llamar al endpoint de listado, la API responde con un arreglo JSON (Figura 6.2).

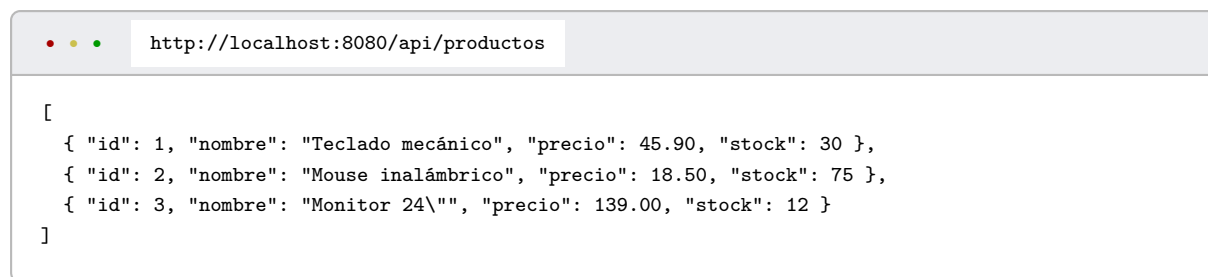


Figura 6.2: Ejecución de la API: GET /api/productos devuelve la lista en formato JSON con código 200.

6.3 Frontend: consumir la API con fetch

Aquí se cierra el círculo con la Unidad I. Una página HTML con JavaScript pide los datos a la API (Figura 6.2) y construye la tabla en el navegador, sin recargar la página.

```

1 <!DOCTYPE html>
2 <html lang="es-EC">
3 <head>
4   <meta charset="UTF-8"><title>Catálogo (cliente)</title>
5   <style>
6     body { font-family: system-ui; max-width: 700px; margin: 2rem auto; }
7     table { border-collapse: collapse; width: 100%; }
8     th, td { border: 1px solid #ccc; padding: 8px; }
9     th { background: #006633; color: #fff; }
10  </style>
11 </head>
12 <body>
13   <h1>Catálogo de productos</h1>
14   <table>
15     <thead><tr><th>ID</th><th>Nombre</th><th>Precio</th><th>Stock</th></tr></thead>
16     <tbody id="cuerpo"><tr><td colspan="4">Cargando.</td></tr></tbody>
17   </table>
18
19   <script>
20     async function cargar() {
21       const resp = await fetch('/api/productos'); // pide datos a la API
22       const productos = await resp.json(); // parsea el JSON
23       const cuerpo = document.getElementById('cuerpo');
24       cuerpo.innerHTML = productos.map(p => `
25         <tr>
26           <td>${p.id}</td>
27           <td>${p.nombre}</td>
28           <td>${p.precio}</td>
29           <td>${p.stock}</td>
30         </tr>`).join('');
31     }
32     cargar(); // al cargar la página
33   </script>

```

```
34 </body>
35 </html>
```

Listing 6.2: static/catalogo.html (cliente)

Al abrir esta página, JavaScript llama a la API y rellena la tabla dinámicamente. La Figura 6.3 muestra el resultado: visualmente parecido al listado de Thymeleaf, pero generado en el navegador a partir de JSON.

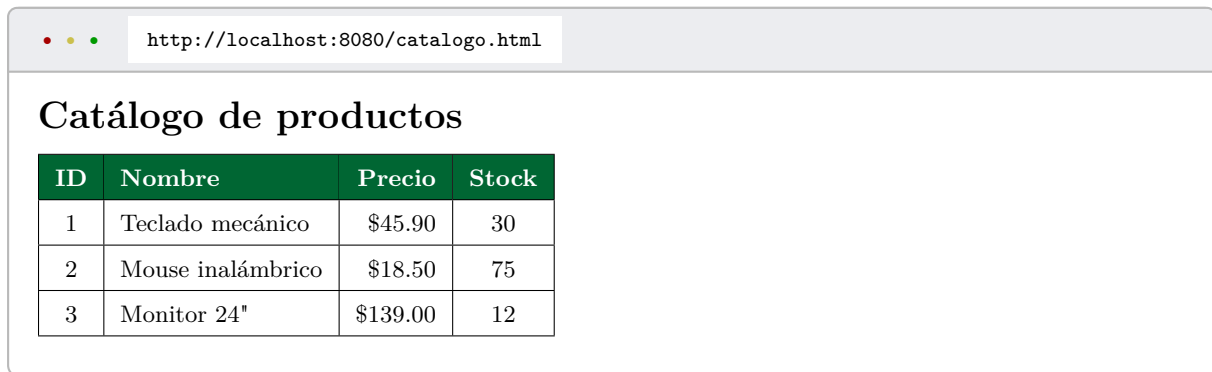


Figura 6.3: Ejecución del cliente: `fetch` obtuvo el JSON de la Figura 6.2 y JavaScript construyó la tabla en el navegador.

Documentación automática con OpenAPI/Swagger

Añadiendo la dependencia `springdoc-openapi-starter-webmvc-ui`, Spring genera documentación interactiva de la API en `/swagger-ui.html`: cada endpoint queda descrito y se puede probar desde el navegador. Es una buena práctica profesional documentar el 100 % de los endpoints.

La pareja Figura 6.2–Figura 6.3 demuestra la separación cliente/servidor que sostiene la web moderna: el mismo backend REST puede alimentar una página web, una app móvil o un programa de terceros.

Capítulo 7

Síntesis, buenas prácticas y ejercicios

«Saber qué modelo usar es tan importante como saber usarlos todos.»

— Nota docente

7.1 Comparativa de los tres modelos

A lo largo de la unidad construimos la misma idea —responder a peticiones— con tres modelos. La Tabla 7.1 los confronta para que el estudiante elija con criterio en cada proyecto.

Tabla 7.1: Comparativa de los tres modelos de Java web.

Criterio	Servlet puro	Servlet + JSP	Spring Boot
Configuración	Manual	Manual	Automática
Vista	HTML en Java	JSP separado	Thymeleaf / REST
Productividad	Baja	Media	Alta
Acceso a datos	JDBC manual	JDBC manual	Spring Data JPA
Validación	A mano	A mano	@Valid declarativa
Uso hoy	Aprendizaje	Mantenimiento	Estándar profesional

La conclusión de la Tabla 7.1 es clara: para proyectos nuevos se usa Spring Boot, pero conocer Servlets y JSP explica cómo funciona por dentro y permite mantener sistemas heredados.

La Figura 7.1 resume el recorrido completo de la unidad como una línea de evolución que culmina en la aplicación construida.

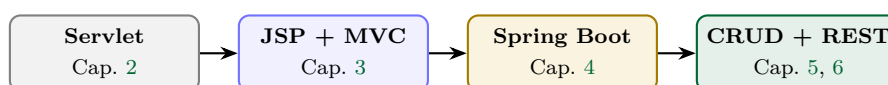


Figura 7.1: Síntesis de la Unidad II: del motor (Servlet) a la aplicación completa (CRUD + REST), pasando por el patrón MVC y el framework moderno.

7.2 Buenas prácticas profesionales

La caja siguiente reúne doce reglas que el estudiante debe respetar en todo proyecto Spring Boot [2].

Doce reglas profesionales 2026

1. Inyección de dependencias por constructor; nunca `new` para servicios.
2. Separar capas: controlador → servicio → repositorio.
3. Validar siempre en el servidor con `@Valid`, además del cliente.
4. Usar DTOs en las APIs para no exponer las entidades directamente.
5. Devolver los códigos HTTP correctos (201 al crear, 404 si no existe).
6. Escapar la salida en las vistas para prevenir XSS (Thymeleaf lo hace).
7. Nunca concatenar SQL: usar consultas parametrizadas o JPA.
8. Aplicar el patrón POST–Redirect–GET en formularios.
9. Externalizar la configuración en `application.properties` por ambiente.
10. Documentar la API con OpenAPI/Swagger.
11. Versionar con Git y escribir pruebas (JUnit, MockMvc).
12. No guardar datos de usuario en campos de instancia de un Servlet/controlador.

7.3 Ejercicios propuestos en IntelliJ IDEA

Ejercicio 7.1 (PROPUESTO) — CRUD de estudiantes

Replicar la aplicación TiendaUTEQ para una entidad **Estudiante** con los campos `cedula`, `nombres`, `correo` y `semestre`. Requisitos: validar que el correo tenga formato válido (`@Email`) y que el semestre esté entre 1 y 10. Incluir las cuatro operaciones CRUD con Thymeleaf y la verificación en la consola H2. Entregar el proyecto en un repositorio Git.

Ejercicio 7.2 (PROPUESTO) — API REST + cliente fetch

Exponer la entidad **Estudiante** del Ejercicio 7.1 como API REST con los verbos de la Tabla 6.1. Construir una página `estudiantes.html` que liste los estudiantes con `fetch` y permita eliminarlos con un botón que invoque `DELETE`. Documentar la API con Swagger y verificar todos los códigos de estado.

Ejercicio 7.3 (PROPUESTO) — Búsqueda

Añadir a TiendaUTEQ un buscador por nombre que use el método `findByNombreContainingIgnoreCase` del repositorio. La vista debe incluir un campo de texto que filtre la tabla al enviar el formulario por `GET`.

7.4 Cierre de la unidad

La Unidad II llevó al estudiante del concepto de servidor a una aplicación web real y funcional en Java. Se comprendió el modelo petición–respuesta (Figura 1.1), se abrió el motor con Servlets, se separaron responsabilidades con MVC y JSP, se adoptó la productividad de Spring Boot y se construyó un CRUD completo con interfaz, validación y persistencia, exponiéndolo además como API REST consumible desde el cliente. Con este recorrido el estudiante cumple el resultado de aprendizaje RAU 2 y queda preparado para la Unidad III, donde estos servicios incorporarán gestión de estado, acceso a datos avanzado y patrones arquitectónicos [7].

Bibliografia

- [1] Paul J. Deitel and Harvey Deitel. *Java How to Program: Early Objects*. Pearson, 11th edition, 2017.
- [2] Robert C. Martin. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017.
- [3] Joseph B. Ottinger, Jeff Linwood, and Dave Minter. *Beginning Hibernate: For Hibernate 5*. Apress, 4th edition, 2017.
- [4] Leonard Richardson and Sam Ruby. *RESTful Web Services*. O'Reilly Media, 2007.
- [5] Laurentiu Spilcă. *Spring Start Here: Learn what you need and learn it well*. Manning Publications, 2024.
- [6] Craig Walls. *Spring in Action*. Manning Publications, 6th edition, 2022.
- [7] Hironori Washizaki, editor. *SWEBOK Guide v4.0: Software Engineering Body of Knowledge*. IEEE Computer Society, 2024.